University of Rajshahi
Department of Computer Science and Engineering
B.Sc. Engg. Part-IV Odd Semester Examination – 2025
Course: CSE4132 (Artificial Intelligence Lab)
Full Marks: 17.50 Time: 03 Hours
Set A

Experiment -1 :

Marks = 6.00

Consider the polynomial equation:

$$y = 5x^3 + 2x^2 - 3x + 7$$

Implement a fully connected neural network (FCNN) to learn the relationship between x and y and predict y for unseen values of x.
Tasks:

a) Design and implement an FCNN with **two hidden layers**. Use ReLU activation in the hidden layers and appropriate activation in the output layer.
b) Generate a dataset by sampling at least 800 values of x from the interval [− 10, 10] and calculating the corresponding y. Split the dataset into training, validation, and testing sets.
c) Train the network using the Adam optimizer and MSE loss. Plot the training and validation loss against epochs.
d) Plot the actual polynomial curve and the predictions made by the trained network.

Experiment -2 : Marks = 6.50

Implement MobileNet based image classification models for the following four CIFAR-10 classes: Frog, Horse, Ship, Truck.
Compare the following three training strategies: (i) Training from scratch, (ii) Transfer learning, (iii) Fine-tuning
Tasks:

a) Prepare the four-class CIFAR-10 dataset and establish a consistent training, validation, and testing protocol for all three models.
b) Implement MobileNet with an MLP classification head consisting of two hidden layers and an output layer.
c) Implement and train the three models using optimization settings.
d) Compare their test accuracy, macro-averaged precision, recall training time, and number of trainable parameters. Present the results in a table.
e) Plot the training and validation accuracy and loss curves for the three approaches.

LAB Viva Marks = 5.00

CamScanner

University of Rajshahi
Department of Computer Science and Engineering
B.Sc. Engineering Part-4 Odd Semester Practical Examination-2025
Course: CSE4142 (Microcontroller Interfacing Lab)

Time: 3 Hours Marks: 25

1. Use 8086 Interfacing Trainer in Kit mode to make a traffic controlling system with three LEDs as the representing light of Red, Green, and Yellow color light. Here the lights will be displayed according to the following sequence and time periods.

   Red light → Yellow light → Green light → Yellow light → Red light

   Red light will be displayed for 10 seconds, Yellow light for 5 seconds and Green light for 15 seconds. The output will be continued until the system is reset.

2. Use 8086 Interfacing Trainer in Kit mode to display letters E, F, and H in an 8x8 dot matrix LED with a delay of 5 seconds between each display. The output will be continued until the system is reset.

3. Use 8086 Interfacing Trainer in Kit mode to display digits 1, 2, and 3 in a 7-Segment LED with a delay of 5 seconds between each display. The output will be continued until the system is reset.

4. **Bidirectional Bare-Metal SPI Half-Duplex Communication Between Two STM32F446xx NUCLEO-F446RE Boards.**

   Objective

   To design and implement a bidirectional, bare-metal SPI half-duplex data transfer protocol between two STM32F446xx NUCLEO-F446RE boards (one configured as Master, the other as Slave) without using HAL libraries.

   The system must satisfy these requirements:

   - When the Master board switch (PC13) is pressed, it transmits a command over the single-wire data line that toggles the Slave board LED (PA5).
   - When the Slave board switch (PC13) is pressed, it signals the Master, swapping the data direction so that a command is sent back to toggle the Master board LED (PA5).

5. **Bare-Metal I2C Half-Duplex Communication Between Two STM32F446xx NUCLEO-F446RE Boards.**

   Objective

   To design and implement a half-duplex bidirectional I2C communication system between two STM32F446xx NUCLEO-F446RE (one configured as Master, the other as Slave) using bare-metal C programming. The system will exchange control signals to toggle on-board LEDs using on-board user push-buttons.

   The communication workflow must satisfy the following requirements:

   - When the Master board's User Switch (PC13) is pressed, the Master sends a data byte (0x01) to the Slave. Upon receiving this byte, the Slave board's User LED (PA5) must blink 3 times.
   - When the Slave board's User Switch (PC13) is pressed, the Slave must wait for the Master to request data. The Master must periodically poll or check the slave (or use an external trigger line). When the Master reads a data byte (0x02) from the Slave, the Master board's User LED (PA5) must blink 3 times.

CamScanner

Department of Computer Science and Engineering
Rajshahi University
B.Sc, Engg. Part-IV Odd Semester Examination-2025
Course: CSE-4192 (Blockchain Lab.)
Time: 3 Hours
[Answer all the questions]

---

1. Consider a blockchain block containing multiple transactions. 7.00
Write a program that calculates the Merkle Root before and after modifying one transaction.

Requirements:

1. Generate the original Merkle Root.
2. Modify the amount or receiver address of a transaction.
3. Recalculate the Merkle Root.
4. Compare both roots.
5. Report whether transaction integrity has been compromised.

2. Develop a simple transaction verification system using digital signatures. 8.00
A transaction consists of the following fields:

- Sender address
- Receiver address
- Transaction amount
- Timestamp

Requirements:

1. Generate a public-private key pair for two users.
2. Allow the sender to create a transaction.
3. Sign the transaction using the sender's private key.
4. Verify the signature before accepting the transaction.
5. Attempt to alter the transaction amount.
6. Reject the modified transaction if signature verification fails.

3. Viva 2.50

CamScanner

University of Rajshahi

Department of Computer Science and Engineering

B.Sc. Engineering 4th Year Odd Semester Exam 2025

Course Code: CSE 4122 Course Title: Object-Oriented Design and Design Pattern

Time: 03 (Three) hours (**Set-Morning**) Full Marks:

Project scenario: A farm currently relies on manual monitoring of soil moisture, irrigation, fertilizer application, and crop health. This causes delayed decisions, unnecessary water consumption, and difficulties responding to changing weather conditions. Develop an **Agricultural Automation System (AAS)** that monitors farm conditions, supports irrigation decisions, and sends notifications to farmers. The system should support different sensor types, alternative irrigation strategies, and integration with an external weather service. Consider the following actors: **Farmer, Farm Administrator, IoT Sensor, Weather Service, Drone System, and AI Analytics System.**

Instructions:

① Complete **Phases 1–4 on paper** and **Phase 5 in Java.**

② Keep class names and responsibilities consistent across your written design and implementation.

③ Use simulated sensor readings and weather data. Actual hardware, internet access, a database, and AI model training are not required.

④ Implement a console application demonstrating the core irrigation workflow.

Phase 1: Requirement Analysis: Analyze the Agricultural Automation System and complete the following tasks:

① Briefly describe the system background and identify one problem with the existing manual farming process.

② State one primary objective and one specific objective of the proposed system.

③ Identify two actors and briefly describe their roles.

Phase 2: Object Identification: Identify and classify **three suitable classes** for the AAS:

1. One Entity class.
2. One Control class.
3. One Boundary class.

Provide at least two attributes for the Entity class and briefly explain the responsibilities of the Control and Boundary classes. Present your answer using the following format:

Class name	Classification	Attributes or responsibilities
Identified class	Entity / Control / Boundary	Relevant details

Phase 3: CRC Card Development : Prepare **two CRC cards** for classes involved in soil monitoring, weather information, irrigation decisions, or farmer notification. Each CRC card must include:

① Class name.

② At least two meaningful responsibilities.

③ Collaborating classes.

The two cards must collectively include at least one Control class and one other suitable class.

Phase 4: UML Modeling: Draw a UML class diagram for the core irrigation subsystem, including at least four classes or interfaces.

Clearly show:

CamScanner

① Class and interface names.

② Important attributes and methods.

③ Visibility indicators (+, −, #).

④ Associations and relevant multiplicities.

⑤ Inheritance or interface realization where appropriate.

⑥ Classes supporting your selected design patterns.

Show aggregation or composition only were justified by the ownership and lifecycle relationships.

Phase 5: Java Implementation: Implement a console-based irrigation subsystem using **all five design patterns** below.

Implement a console-based irrigation subsystem using the following five design patterns.

Pattern	Required implementation
Singleton	Implement FarmDataManager with a private constructor and controlled access method. Store simulated farm information and demonstrated that two access calls return the same instance.
Factory	Implement SensorFactory to create at least two sensor types, such as SoilMoistureSensor and TemperatureSensor, through a common Sensor interface. Handle unsupported sensor types. A Simple Factory is acceptable; identify the variant used.
Observer	Implement an alert publisher and two subscribers, such as FarmerNotifier and FarmDashboard. Demonstrate notification to both observers.
Strategy	Implement an IrrigationStrategy interface with NormalIrrigationStrategy and WaterSavingStrategy. Allow the controller to switch strategies at runtime.
Adapter	Implement WeatherService and a WeatherAPIAdapter that wraps a simulated external weather class with an incompatible method or data format.

Required irrigation rules: Use these simplified examination rules rather than treating them as agricultural recommendations.

1. Normal strategy: Irrigate when soil moisture is below 40% and rain is not expected.
2. Water-saving strategy: Irrigate when soil moisture is below 25% and rain is not expected.
3. Otherwise, leave the pump off.
4. Reject soil-moisture readings outside 0–100%.

The irrigation controller must obtain weather information through the adapter, apply the selected strategy, control the simulated pump, and publish the irrigation decision to registered observers.

Required test demonstrations: Demonstrate the following six test cases through your console application:

Test	Input or action	Expected behavior
1	Moisture 30%, no rain, normal strategy	Pump on
2	Moisture 30%, no rain, water-saving strategy	Pump off
3	Moisture 20%, rain expected, either strategy	Pump off
4	Moisture −5% or 105%	Invalid reading rejected; pump not activated
5	Two observers registered, then publish an alert	Both observers receive the alert
6	Request FarmDataManager twice	Both references identify the same instance

Note: Sensor creation for both supported sensor types and handling of an unsupported type must also be demonstrated as part of the Factory implementation. Demonstrate observer unsubscription through a brief explanation or an additional console operation.

CamScanner

Roll : 2211176109

University of Rajshahi
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
B. Sc. Engineering Part IV Odd Semester Exam-2026
C. Code: CSE4111 Title: Parallel Processing and Distributed System Lab

Time: 03 Hours Full Marks: 17.5

Set-A

Problem 1: Design a system where one MPI process (Master) distributes array elements among multiple worker processes. Each worker calculates the sum of its assigned elements and sends the partial sum back to the Master using point-to-point communication.

Sample Input	Sample Output
Number of processes: 4 • P0 = Master • P1 = Worker • P2 = Worker • P3 = Worker Number of elements = 9 Array: 5 10 15 20 25 30 35 40 45	P0 distributed elements to workers. P1 calculated partial sum: 30 P2 calculated partial sum: 75 P3 calculated partial sum: 120 P1 sent partial sum: 30 to P0 P2 sent partial sum: 75 to P0 P3 sent partial sum: 120 to P0 P0 received partial sums from all workers. Total sum of array elements = 225

Problem 2: Each MPI process stores a local integer value. First, calculate the square of the local value. Then, use MPI_Scan with the MPI_SUM operation to calculate the prefix sum of squares across all processes. Compare the result with a manually implemented point-to-point solution.

Sample Input	Sample Output
Number of processes: 4 • P0 = 2 • P1 = 3 • P2 = 4 • P3 = 5	P0: Local value = 2, Square = 4, Prefix sum = 4 P1: Local value = 3, Square = 9, Prefix sum = 13 P2: Local value = 4, Square = 16, Prefix sum = 29 P3: Local value = 5, Square = 25, Prefix sum = 54 Prefix sum of squares: 4 13 29 54

Problem 3: Viva Voce

CamScanner